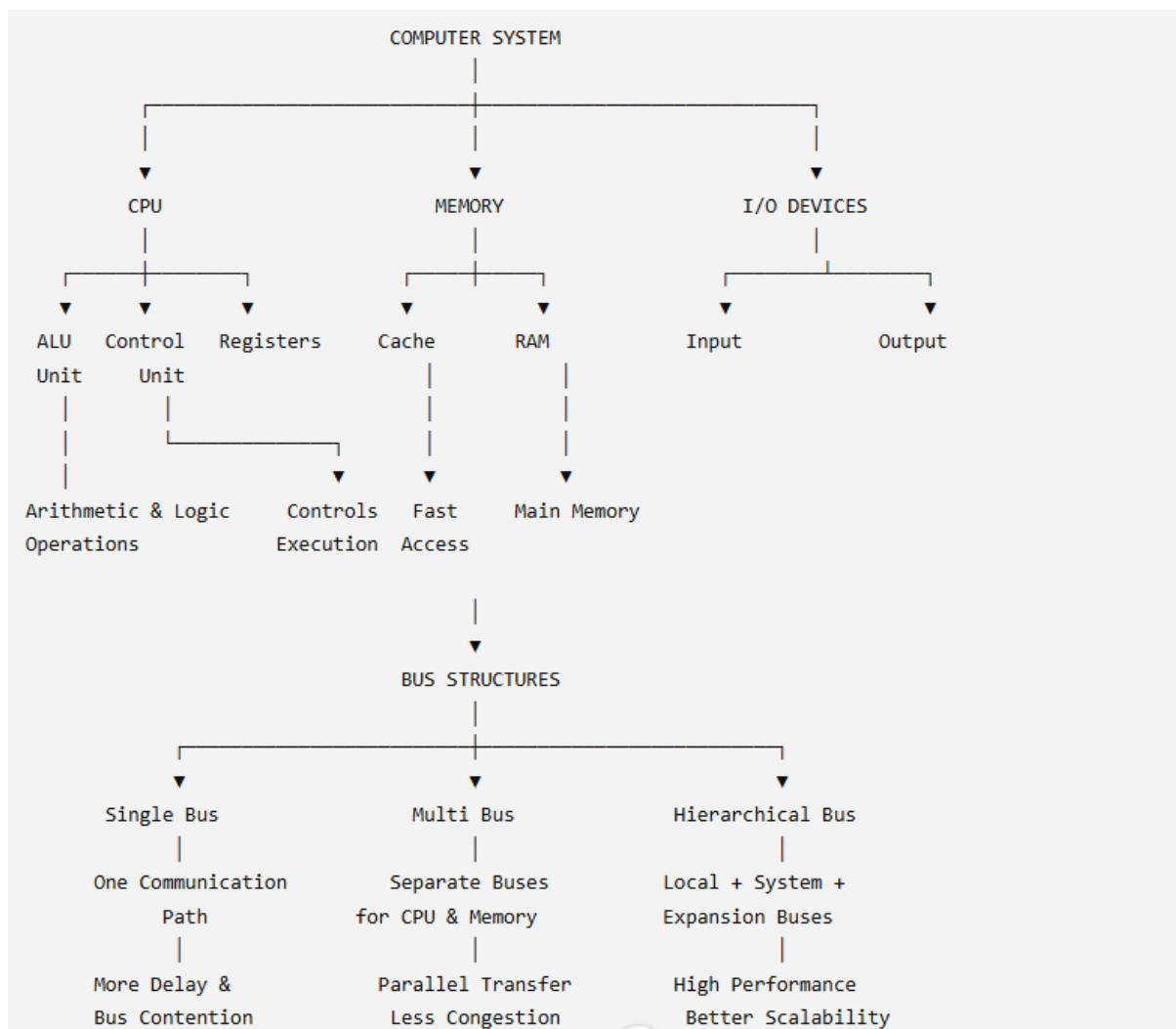


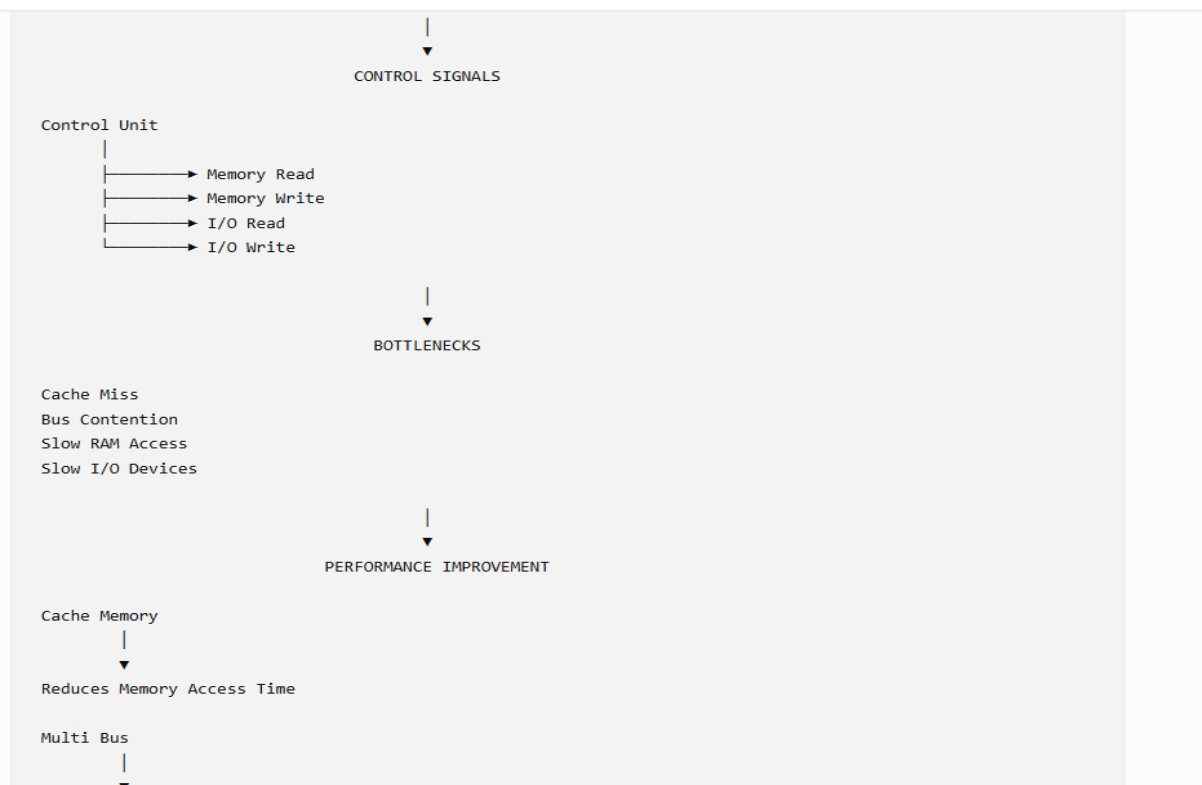
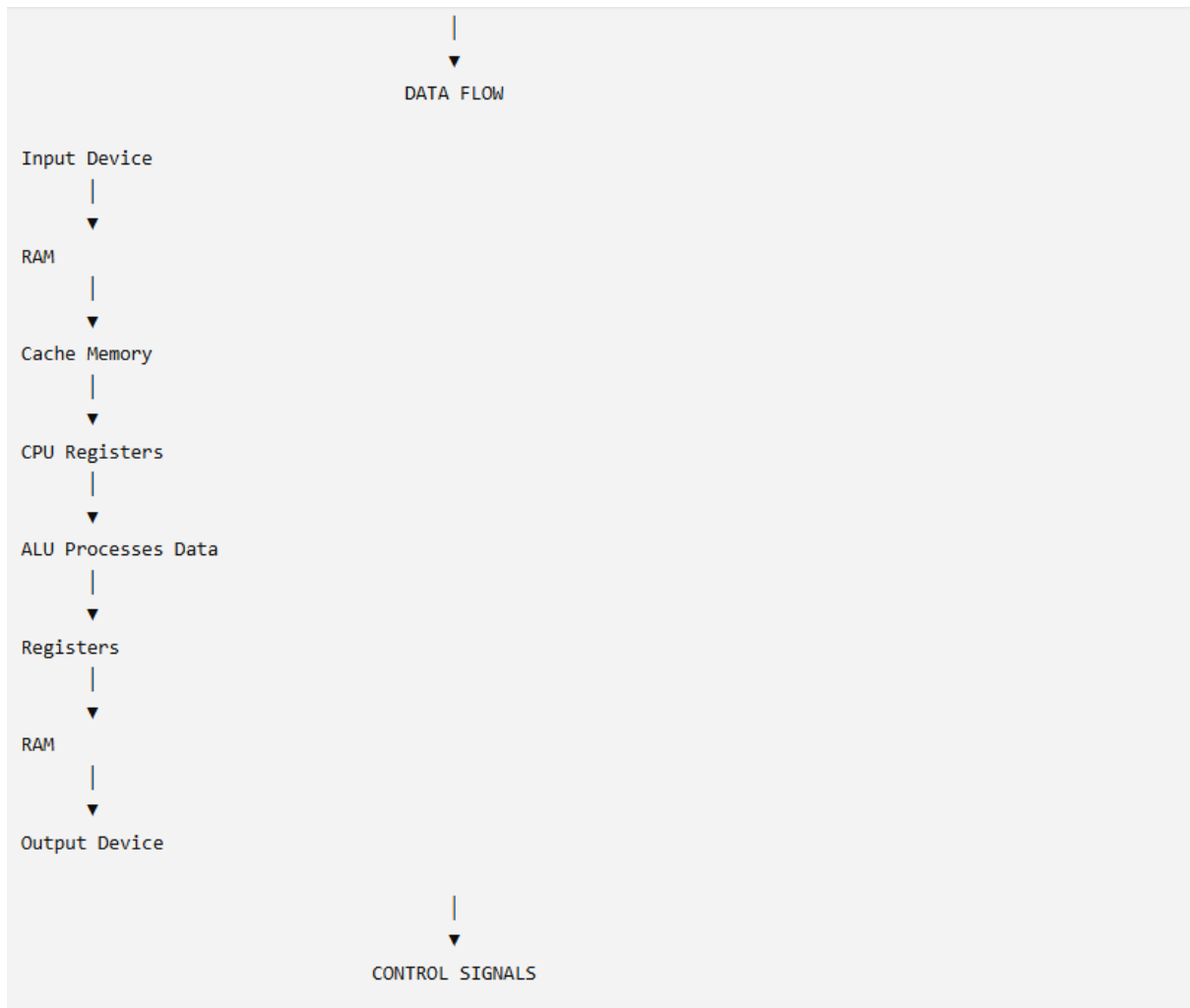
Q1. Concept Map Showing the Relationship Between CPU, Memory, I/O Devices and Bus Structures

Introduction

A computer system consists of three major components: the **Central Processing Unit (CPU)**, **Memory**, and **Input/Output (I/O) devices**. The CPU acts as the brain of the computer by processing instructions, while memory stores data and programs required for execution. Input devices provide data to the system, and output devices display the processed results. These components communicate through different bus structures such as **single bus**, **multi-bus**, and **hierarchical bus**, which transfer data, addresses, and control signals efficiently. The choice of bus structure directly affects the overall performance of the computer system by reducing delays and improving communication.

Concept Map





Conclusion

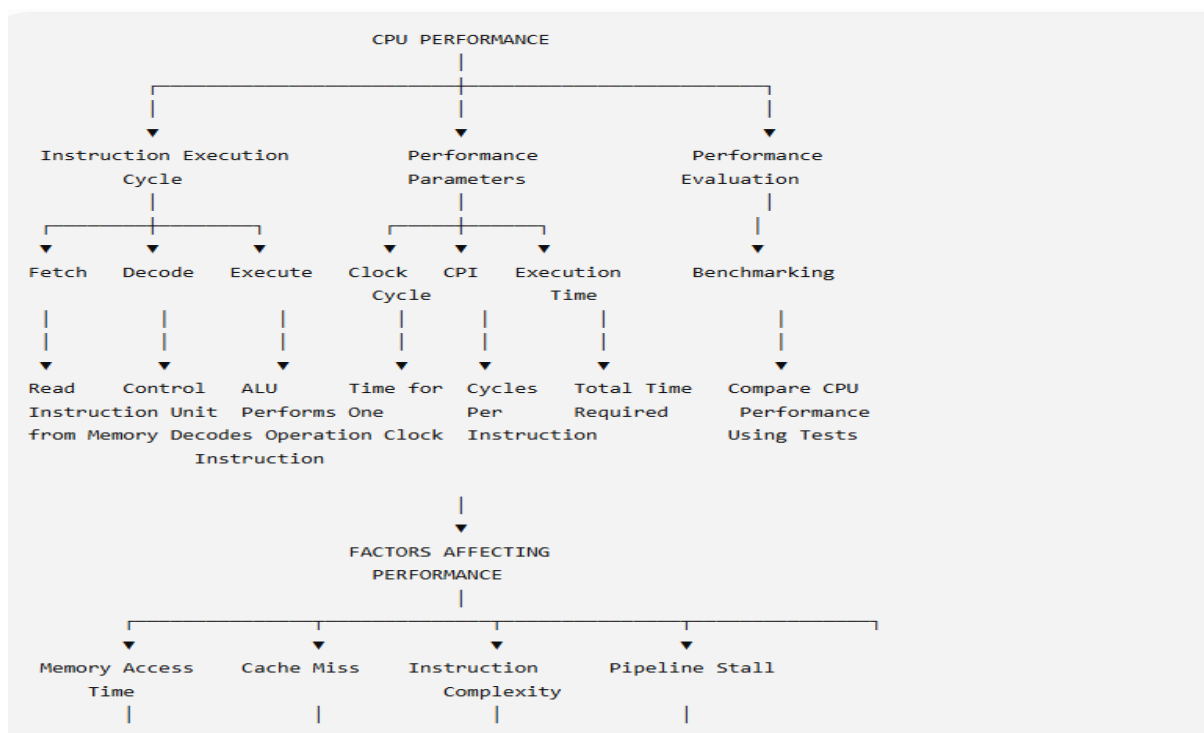
The CPU, memory, and I/O devices work together to execute every instruction in a computer system. Efficient communication between these components is achieved using bus structures. A **single bus** is simple but may suffer from delays due to heavy traffic, whereas **multi-bus** and **hierarchical bus** architectures improve performance by enabling parallel communication and reducing bottlenecks. The use of cache memory, optimized bus structures, and proper control signals ensures faster execution, lower memory access time, and improved overall system efficiency.

Q2. Concept Map Linking the Instruction Execution Cycle with CPU Performance Parameters

Introduction

The **Instruction Execution Cycle** is the sequence of steps followed by the CPU to execute every instruction in a program. It consists of three main stages: **Fetch, Decode, and Execute**. During the fetch stage, the CPU retrieves an instruction from memory. In the decode stage, the Control Unit interprets the instruction, and in the execute stage, the ALU or other CPU components perform the required operation. The overall performance of the CPU depends on several factors such as **Clock Cycles, Cycles Per Instruction (CPI), Execution Time, Memory Access Time, Instruction Complexity, Pipeline Stalls, and Benchmarking**. Understanding these relationships helps in designing faster and more efficient computer systems.

Concept Map





Conclusion

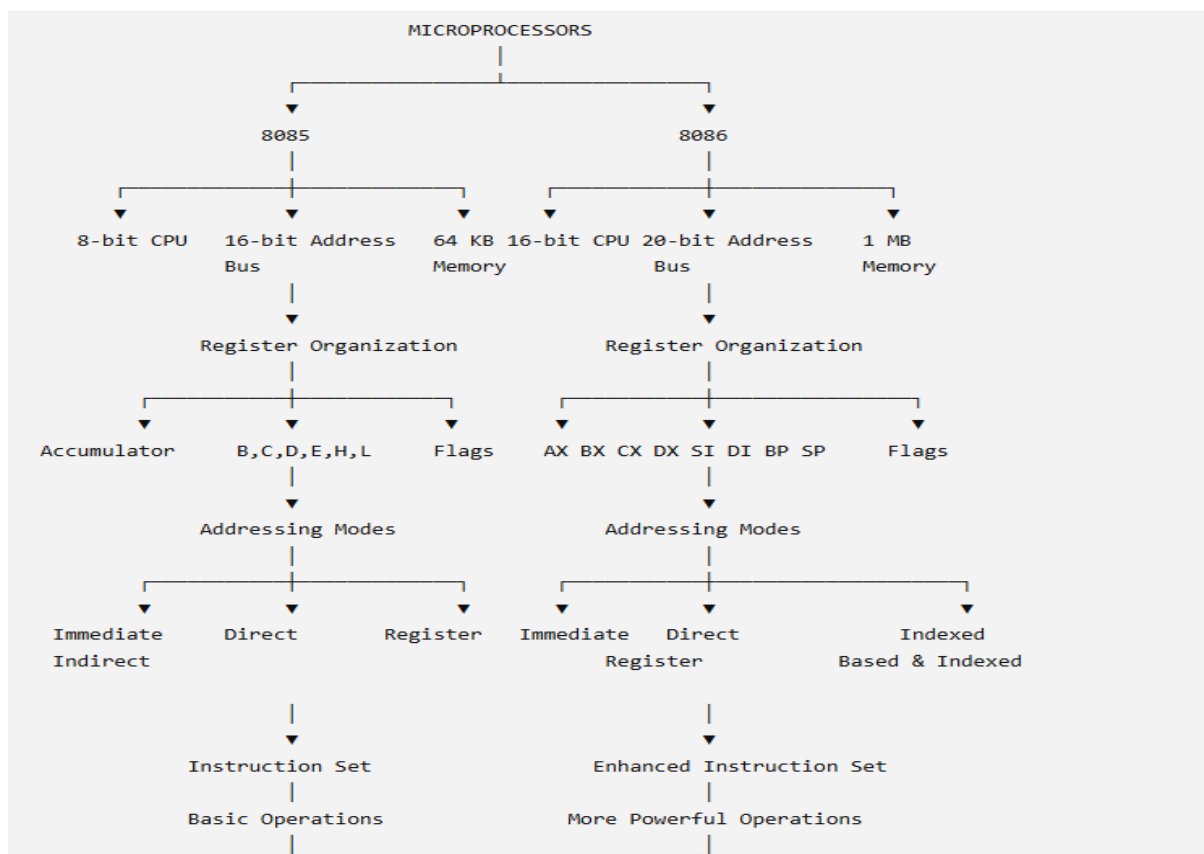
The instruction execution cycle is the foundation of CPU operation, consisting of the **Fetch, Decode, and Execute** stages. Each stage contributes to the total execution time of a program. Performance parameters such as **Clock Cycle, CPI, and Execution Time** are directly influenced by memory access delays, instruction complexity, and pipeline stalls. Benchmarking helps compare different processors and evaluate their efficiency under standard workloads. By improving cache performance, reducing CPI, and optimizing pipelines, modern processors achieve higher speed, better efficiency, and improved overall system performance.

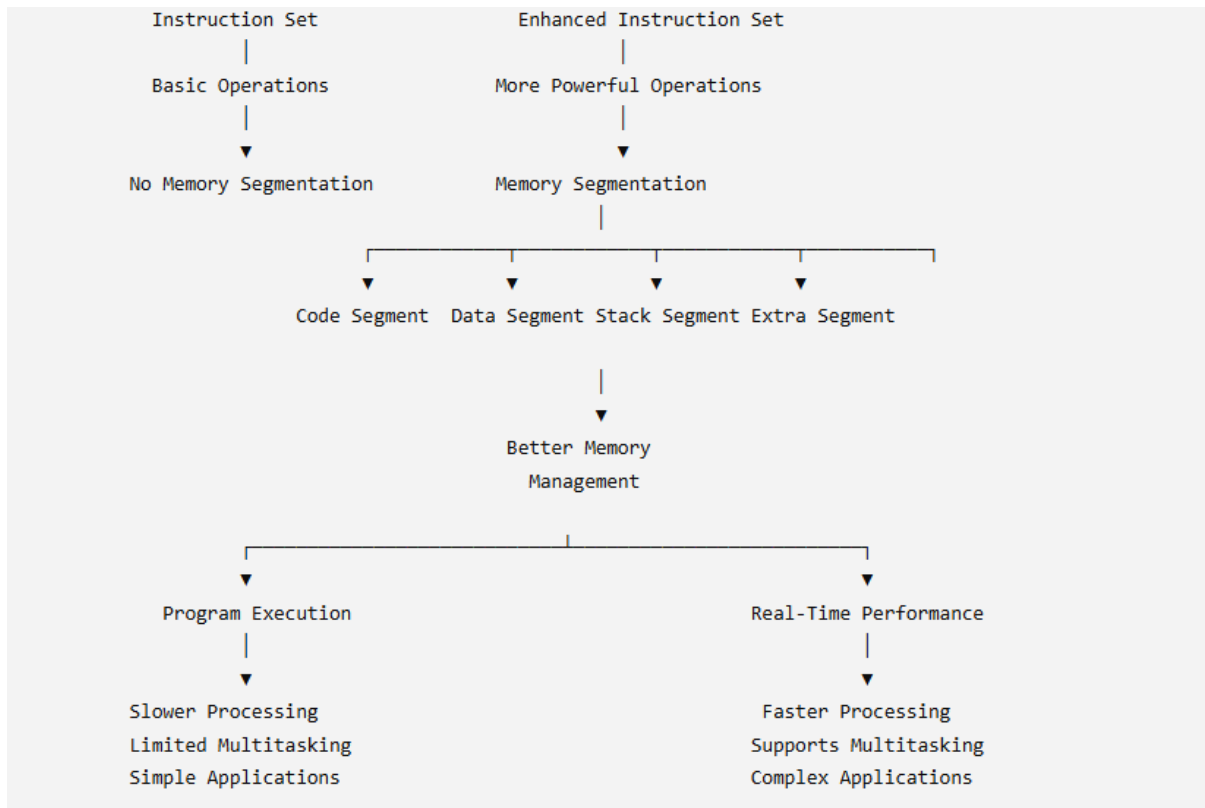
Q3. Concept Map Comparing 8085 and 8086 Architectures

Introduction

The **8085** and **8086** are two important microprocessors developed by Intel, widely used in the evolution of computer architecture. The **8085** is an **8-bit microprocessor** designed for simple computing applications, whereas the **8086** is a more advanced **16-bit microprocessor** with higher processing speed, larger memory capacity, and improved architecture. Both processors differ in terms of **instruction set**, **register organization**, **addressing modes**, **memory organization**, and **execution capability**. The introduction of **memory segmentation** in the 8086 allows efficient memory management and supports multitasking applications. Comparing these processors helps in understanding the advancement of modern processor design and its impact on system performance.

Concept Map





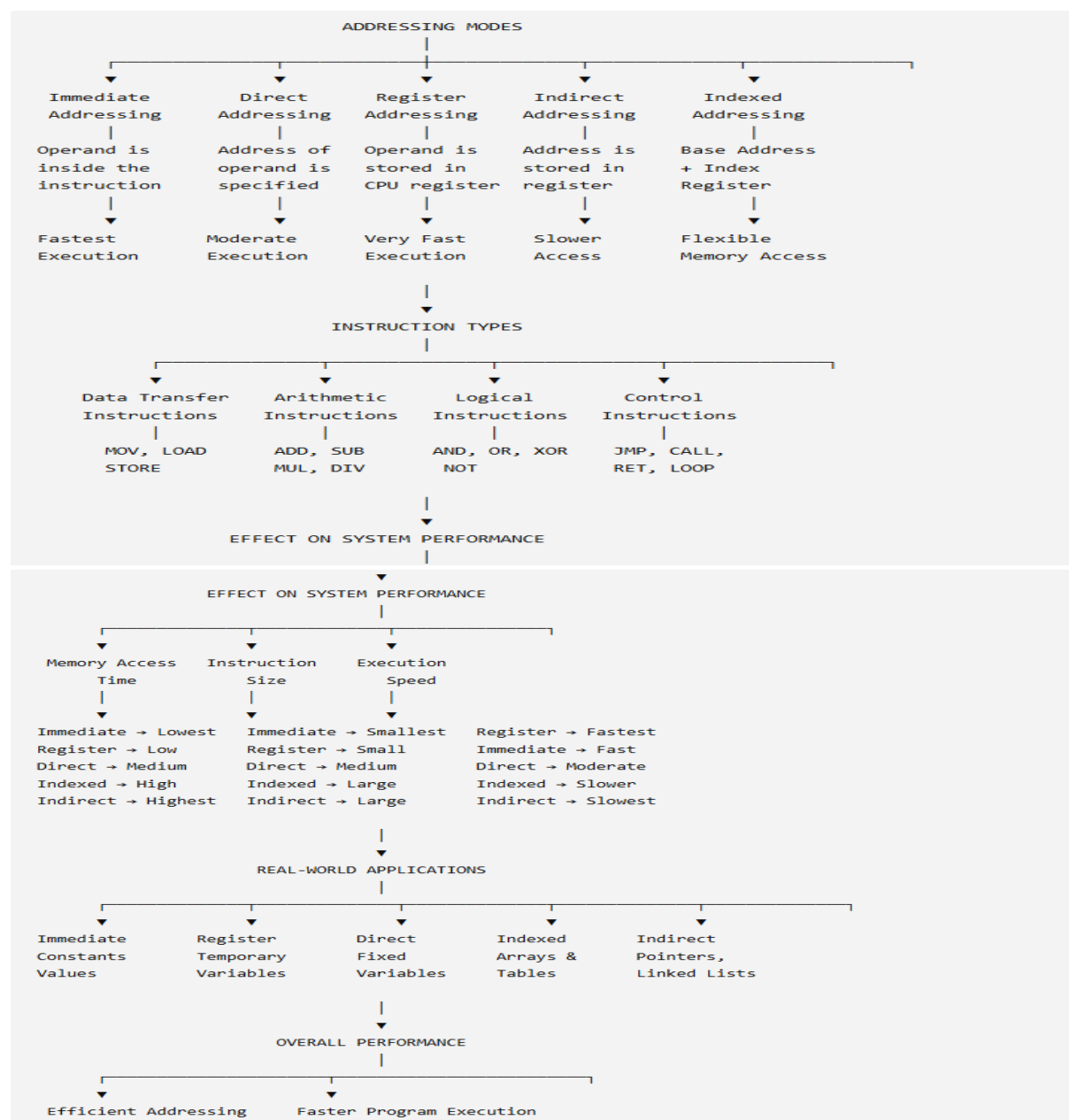
Conclusion

The **8085** microprocessor is suitable for simple and basic computing tasks because of its **8-bit architecture and smaller memory capacity**. In contrast, the **8086** provides **16-bit processing, a larger address space, memory segmentation, and more powerful instructions**, making it suitable for advanced and real-time applications. The architectural improvements in the 8086 result in faster execution, efficient memory utilization, better multitasking capability, and higher overall system performance. Therefore, the 8086 represents a significant advancement over the 8085 and forms the foundation for modern microprocessor architectures.

Q4. Concept Map Connecting Addressing Modes with Instruction Types

Introduction

Addressing modes define the method by which a CPU locates the operands required for executing an instruction. They play an important role in determining the **memory access time**, **instruction size**, **execution speed**, and **overall processor performance**. Different addressing modes are suitable for different programming situations. For example, **Immediate Addressing** is used when data is directly available within the instruction, while **Indexed Addressing** is useful for accessing arrays and tables. Addressing modes are closely associated with various instruction types such as **Data Transfer, Arithmetic, Logical, and Control Instructions**, enabling efficient execution of programs in computer architecture.



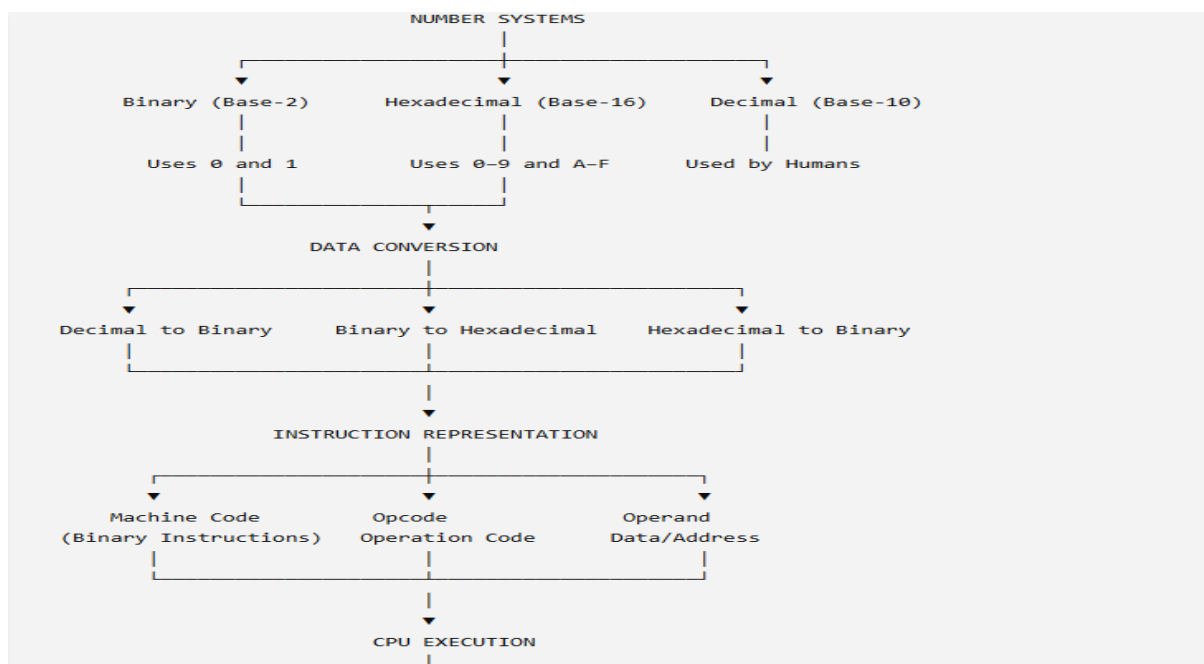
Conclusion

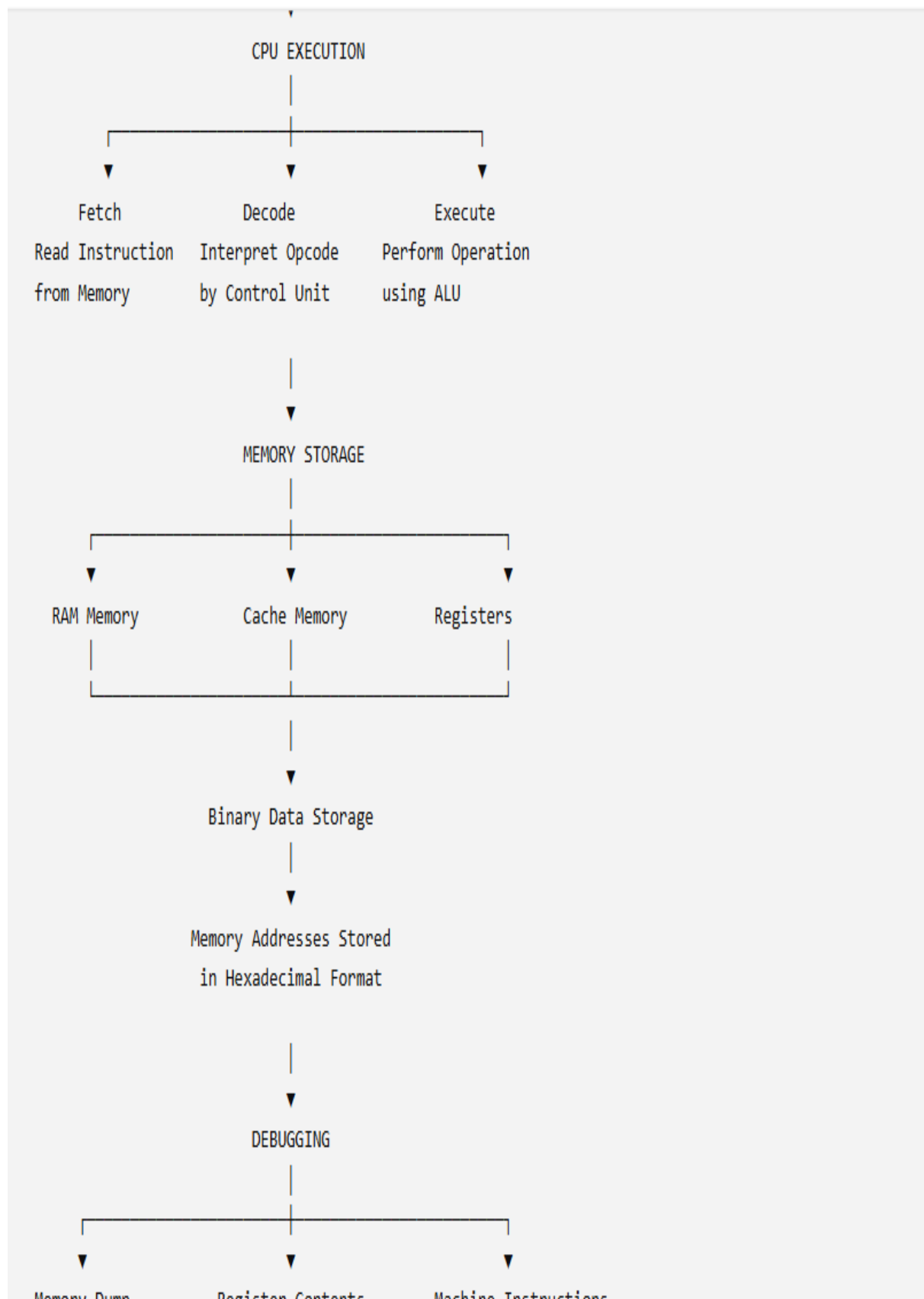
Addressing modes are an essential part of computer architecture because they determine **how operands are accessed during instruction execution**. Each addressing mode has its own advantages depending on the application. **Register and Immediate Addressing** provide faster execution due to minimal memory access, whereas **Direct, Indexed, and Indirect Addressing** offer greater flexibility for handling complex data structures. Selecting the appropriate addressing mode improves execution speed, reduces memory access time, and enhances the overall efficiency of the processor.

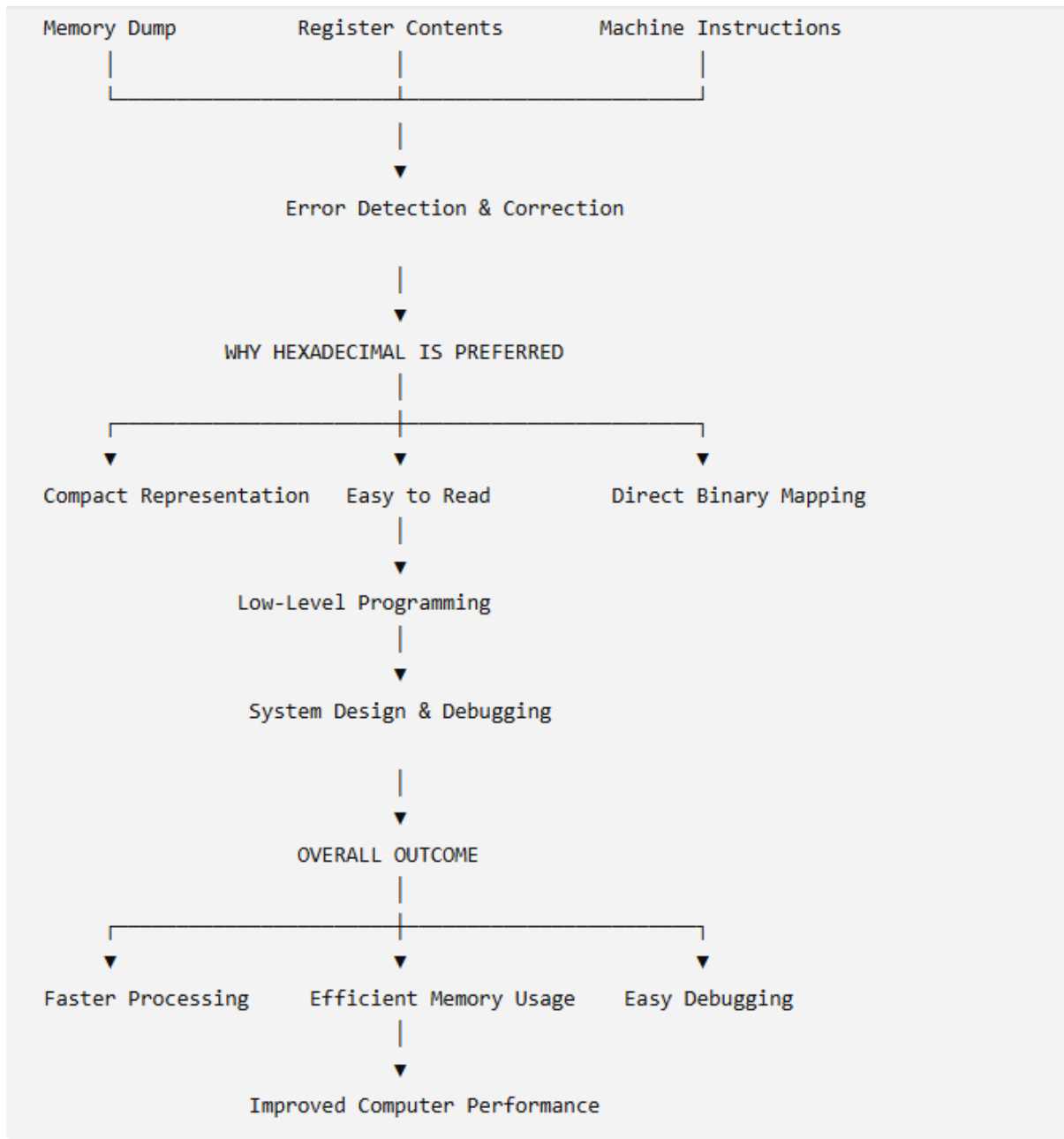
Q5. Concept Map Integrating Number Systems with Instruction Representation, Memory Storage, and Debugging

Introduction

Number systems form the foundation of digital computers because all data and instructions are represented using binary digits (0 and 1). The two most commonly used number systems in computer architecture are **Binary (Base-2)** and **Hexadecimal (Base-16)**. While the CPU processes information in binary form, hexadecimal representation provides a compact and human-readable way to represent binary values. Number systems are closely related to **instruction representation, memory storage, machine language programming, and debugging processes**. Understanding these relationships helps programmers and system designers efficiently develop, analyze, and troubleshoot computer systems.







Conclusion

Number systems are fundamental to the operation of every computer system. Although the CPU internally processes data in **binary**, programmers often use **hexadecimal** because it is easier to read, write, and debug. Instructions are stored in binary format within memory and interpreted by the CPU during the **fetch, decode, and execute** cycle. Memory addresses, machine instructions, and register contents are commonly represented in hexadecimal to simplify low-level programming and system analysis. Thus, the integration of **number systems, instruction representation, memory storage, and debugging** ensures accurate execution of programs and improves the overall efficiency and reliability of modern computer systems.